

OPCVM GSM

ARCHITECTURE DES BASES DE DONNÉES ET PERSISTANCE DES INTERFACES

Gestion sous mandat, espace client, marchés, ordres, liquidité, Cession_Retrait et contrôle

Document d'architecture fonctionnelle et technique
Version 1.0 - 14 septembre 2026

Objectif du document

Définir une architecture de données durable permettant à chaque interface du logiciel d'afficher, créer, modifier et historiser des données réelles, tout en garantissant l'intégrité des portefeuilles, la traçabilité des opérations et la séparation entre données métier et données de marché.

1. Résumé exécutif

La maquette OPCVM GSM couvre déjà un périmètre fonctionnel proche d'un véritable système de gestion de portefeuille : clients, profils de risque, allocations, positions, ordres, avis d'opéré, liquidité, revenus, marchés actions et obligataires, analyse, alertes, rééquilibrage, recommandations, comité d'investissement, Cession_Retrait et Cession interne. Pour passer d'une maquette alimentée par des constantes JavaScript à une application opérationnelle, la donnée doit être déplacée vers des sources persistantes et transactionnelles.

L'architecture recommandée repose sur deux bases SQL principales, une zone d'intégration, Redis pour les fonctions temporaires et un stockage documentaire. Le frontend ne doit jamais écrire directement dans les tables : il exprime une intention via une API métier Django/DRF, puis le backend contrôle les règles, verrouille les ressources nécessaires et applique l'opération de façon atomique.

Principe directeur

Une seule source de vérité pour chaque donnée. Les écrans peuvent être nombreux, mais une position, un solde espèces, un ordre ou une demande de retrait ne doit jamais exister sous plusieurs versions indépendantes.

Architecture cible

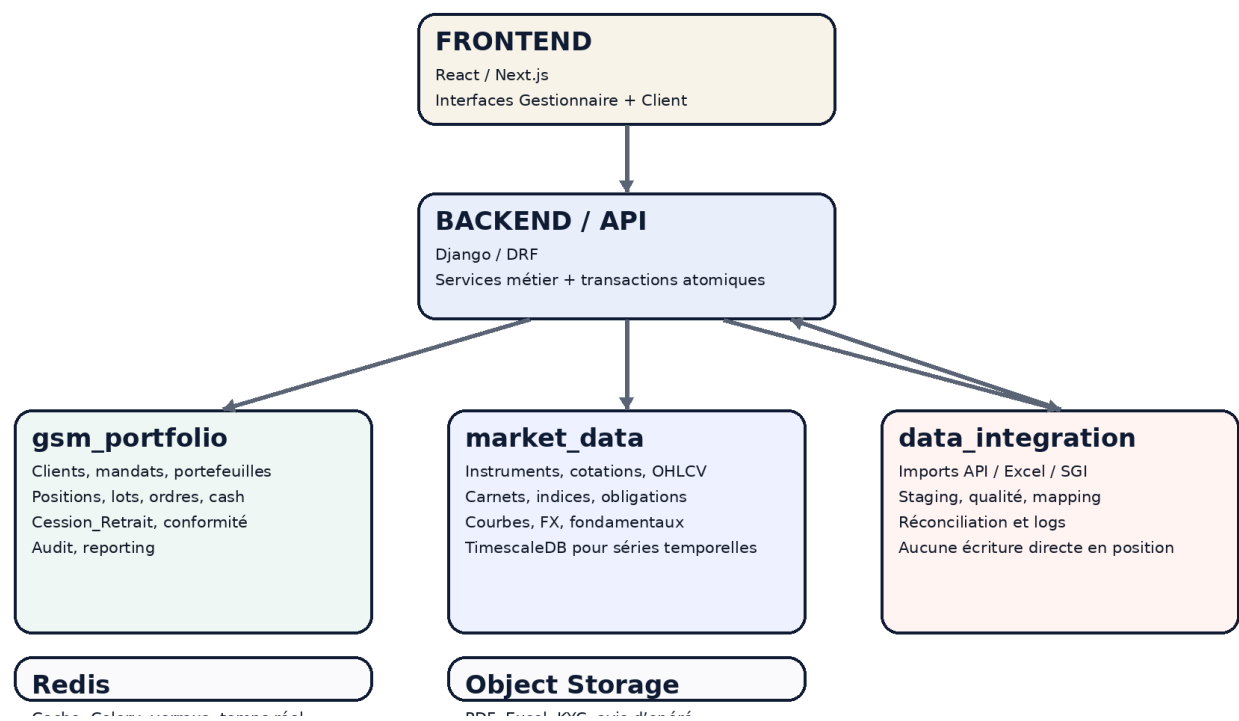


Figure 1 - Architecture logique proposée.

Les cinq briques de stockage

Brique	Technologie recommandée	Responsabilité
gsm_portfolio	PostgreSQL	Données métier et transactionnelles : clients, portefeuilles, positions, ordres, cash, retraits, conformité, audit.
market_data	PostgreSQL + TimescaleDB	Référentiel instruments, cotations, OHLCV, carnet de marché, indices, obligations,

Brique	Technologie recommandée	Responsabilité
		courbes, FX, fondamentaux.
data_integration	PostgreSQL / schéma staging	Imports API/Excel/SGL, données brutes, contrôles qualité, mapping et réconciliation.
Redis	Redis	Cache, files Celery, verrous distribués, sessions, calculs temporaires, notifications temps réel.
Object Storage	S3 / MinIO / Azure Blob	PDF, Excel, KYC, contrats, avis d'opéré, rapports et pièces justificatives.

2. Principes de conception

1. Le frontend React/Next.js ne se connecte jamais directement à PostgreSQL.
2. Toute modification passe par une API métier contrôlée par Django/DRF.
3. Les opérations financières sensibles utilisent des transactions SQL atomiques et des verrous adaptés.
4. Les positions sont la conséquence des transactions et lots ; elles ne doivent pas être saisies arbitrairement.
5. Les données de marché sont séparées des données clients afin de maîtriser le volume et les fréquences de mise à jour.
6. Les imports externes passent par une zone de staging et de réconciliation avant d'alimenter les données officielles.
7. Les données calculées peuvent être mises en cache, mais la source de vérité reste en base SQL.
8. Chaque action sensible doit laisser une trace dans audit_logs avec utilisateur, date, objet, état avant/après et corrélation de requête.
9. Les règles telles que retrait maximal 95 %, minimum de clôture, frais ou tolérances doivent être paramétrables et non codées en dur dans l'interface.

3. Base principale : gsm_portfolio

Cette base est le cœur transactionnel. Elle contient tout ce qui appartient à la SGI, aux clients, aux portefeuilles et aux opérations. Les écrans Gestion sous mandat et Espace client lisent principalement cette base à travers les API métier.

Utilisateurs et sécurité

Table	Rôle
users	Comptes utilisateurs, identité applicative, état du compte.
roles	Rôles fonctionnels : ADMIN, PORTFOLIO_MANAGER, INVESTOR, etc.
user_roles	Relation utilisateurs ↔ rôles.
permissions	Droits fins si nécessaire.
user_sessions	Sessions applicatives / terminaux si elles sont persistées.

Organisation SGI

Table	Rôle
sgis	Sociétés de gestion / SGI.
portfolio_managers	Gestionnaires de portefeuille.
manager_client_assignments	Association gestionnaire ↔ client, avec période de validité.
manager_portfolio_assignments	Association gestionnaire ↔ portefeuille.

Clients

Table	Rôle
clients	Fiche maître client : personne/entité, pays, statut, devise de référence.
client_contacts	Téléphone, email, adresse et préférences de contact.
client_bank_accounts	Comptes bancaires de règlement.
client_documents	KYC, pièces, contrats et références documentaires.
client_status_history	Historique des statuts client.

Profils et mandats

Table	Rôle
investor_profiles	Profil de risque courant.
risk_profile_history	Historisation des changements de profil.
investment_constraints	Contraintes client/mandat : exclusions, limites, concentration.
mandates	Contrats de gestion sous mandat.
mandate_rules	Règles propres au mandat.
portfolio_target_allocations	Allocations cibles par portefeuille et classe d'actifs.

Portefeuilles et positions

Table	Rôle
portfolios	Portefeuille principal : client, devise, marché, statut, mandat.
portfolio_accounts	Comptes titres / sous-comptes.
portfolio_cash_accounts	Comptes espèces et soldes.
positions	Position agrégée courante par portefeuille et instrument.
position_lots	Lots d'acquisition servant au CMP, fiscalité et traçabilité.
portfolio_allocations	Allocation réelle par classe/secteur/pays.
portfolio_valuations	Valorisation du portefeuille à une date.
portfolio_performance_snapshots	TWR/MWR et métriques de performance historiques.
portfolio_benchmarks	Association portefeuille ↔ benchmark.

Ordres, exécutions et règlement

Table	Rôle
orders	Ordres saisis depuis le carnet ou le ticket d'ordre.
order_executions	Exécutions partielles ou totales.
order_status_history	Historique du cycle de vie de l'ordre.
order_allocations	Ventilation d'une exécution entre portefeuilles.
transactions	Achats, ventes, transferts, écritures de titres.
transaction_fees	Frais, commissions et taxes.
settlements	Règlement/livraison.
settlement_instructions	Instructions bancaires / dépositaires.
trade_confirmations	Avis d'opéré.
trade_confirmation_documents	Documents PDF associés.

Liquidité et revenus

Table	Rôle
cash_movements	Dépôts, retraits, achats, ventes, coupons, dividendes, frais.
cash_reservations	Liquidité bloquée ou réservée.
expected_cashflows	Flux futurs attendus.
liquidity_snapshots	Photographie de liquidité à une date.
liquidity_sources	Origines des fonds du tableau Money Management.
liquidity_allocations	Affectations / blocages de la liquidité.
dividend_events	Évènements dividendes.
coupon_events	Évènements coupons.
amortization_events	Amortissements d'obligations.
portfolio_income_events	Droits de revenus attribués aux portefeuilles.

Cession_Retrait et Cession interne

Table	Rôle
withdrawal_requests	Demande initiale de retrait.
withdrawal_status_history	Historique du statut du retrait.
withdrawal_plans	Plan de cession optimisé.
withdrawal_plan_lines	Lignes titre/quantité/montant du plan.
withdrawal_plan_versions	Versions après modification du gestionnaire.
withdrawal_constraints	Résultats des règles 95 %, solde minimum, etc.
internal_transfer_searches	Recherche de contreparties internes.
internal_transfer_candidates	Clients acheteurs évalués.
internal_transfer_matches	Matching vendeur ↔ acheteur.

Table	Rôle
internal_transfers	Cession interne validée.
internal_transfer_lines	Détail des instruments et quantités transférés.
internal_transfer_settlements	Règlement de la cession interne.
withdrawal_payments	Paieement final du retrait.
payment_status_history	Historique du paiement.

Pilotage, risque et conformité

Table	Rôle
watchlists	Listes de surveillance.
watchlist_items	Instruments d'une watchlist.
investment_recommendations	Recommandations actions.
recommendation_targets	Objectifs, sens, horizon, prix cibles.
allocation_recommendations	Recommandations d'allocation.
allocation_criteria	Critères et scores utilisés.
rebalancing_runs	Simulation ou exécution d'un rééquilibrage.
rebalancing_lines	Actions à acheter/vendre.
rebalancing_approvals	Validations du plan.
portfolio_risk_snapshots	Volatilité, concentration, drawdown, etc.
stress_scenarios	Scénarios de stress paramétrés.
stress_test_results	Résultats de stress.
correlation_snapshots	Matrices/corrélations historisées.
alert_rules	Règles d'alerte.
alerts	Alertes générées.
alert_status_history	Cycle de vie des alertes.
compliance_rules	Règles réglementaires/mandat.
compliance_checks	Contrôles effectués.
compliance_breaches	Dépassements détectés.
compliance_waivers	Déroations/approbations.
investment_committees	Sessions de comité.
committee_reports	Rapports associés.
committee_decisions	Décisions prises.
committee_decision_impacts	Suivi de l'impact des décisions.
benchmarks	Référentiel des benchmarks.
reports	Définition des rapports.
report_runs	Exécutions de génération.
report_documents	Fichiers générés.
system_parameters	Paramètres : seuils, frais, tolérances, minimum compte.

Table	Rôle
audit_logs	Journal d'audit.
notifications	Notifications utilisateur.
documents	Registre documentaire générique.

4. Tables critiques et champs recommandés

Les tableaux suivants précisent les colonnes minimales des objets qui déterminent directement les soldes, CMP, positions, ordres et retraits. Les types exacts seront ajustés lors de l'implémentation Django/PostgreSQL.

Table	Champs clés	Responsabilité
clients	id (UUID), client_code, client_type, legal_name/full_name, country_id, reference_currency, status, opened_at, closed_at, created_at, updated_at	Point d'entrée de tous les portefeuilles.
portfolios	id, client_id, mandate_id, manager_id, portfolio_code, name, currency, market_scope, status, opened_at, closed_at	Un client peut posséder plusieurs portefeuilles.
portfolio_cash_accounts	id, portfolio_id, currency, ledger_balance, reserved_balance, available_balance, updated_at	Le disponible doit être dérivé du ledger et des réservations.
instruments	id, isin, ticker, name, asset_type, issuer_id, exchange_id, currency, active	Référentiel partagé avec market_data, via ID stable.
positions	id, portfolio_id, instrument_id, quantity, blocked_quantity, available_quantity, average_cost, cost_basis, updated_at	Vue agrégée courante ; recalculable depuis les lots.
position_lots	id, portfolio_id, instrument_id, acquisition_transaction_id, acquisition_date, quantity_initial, quantity_remaining, acquisition_price, fees_allocated, lot_cost	Source de vérité du CMP et du coût historique.
orders	id, portfolio_id, instrument_id, side, order_type, quantity, limit_price, currency, status, validity, created_by, created_at	Créé par l'interface, soumis aux contrôles pré-trade.
order_executions	id, order_id, execution_ref, quantity, price, fees, executed_at, venue	Une commande peut avoir plusieurs exécutions.
transactions	id, portfolio_id, instrument_id, transaction_type, quantity, gross_amount, net_amount, currency, trade_date, settlement_date, execution_id	Écriture métier issue d'une exécution ou d'un transfert.
cash_movements	id, cash_account_id, movement_type, amount, currency, value_date, transaction_id, withdrawal_id, status	Ledger espèces ; éviter de modifier un solde sans mouvement.
withdrawal_requests	id, portfolio_id, amount, currency, requested_at, desired_date, payment_method, status, created_by	Déclencheur du workflow Cession_Retrait.
withdrawal_plans	id, withdrawal_request_id, version, strategy, gross_sale_required, net_sale_required, internal_coverage, market_residual, status	Plan calculé puis éventuellement modifié.
withdrawal_plan_lines	id, plan_id, instrument_id, proposed_quantity, manager_quantity, reference_price, expected_proceeds, fees, liquidity_score	Lignes de cession.
internal_transfers	id, withdrawal_request_id, seller_portfolio_id,	En-tête de cession interne.

Table	Champs clés	Responsabilité
	buyer_portfolio_id, status, matched_amount, trade_date, settlement_date, approved_by	
internal_transfer_lines	id, internal_transfer_id, instrument_id, quantity, price, gross_amount, fees	Chaque ligne affecte les deux portefeuilles.
audit_logs	id, actor_id, action, entity_type, entity_id, correlation_id, before_json, after_json, ip_address, created_at	Indispensable pour la traçabilité et les contrôles.

5. CMP et +/- Value : modèle de calcul réel

L'interface présente désormais le CMP et la plus/moins-value par ligne d'actif. En production, ces valeurs ne doivent jamais être saisies ou simulées : elles découlent des lots d'acquisition et du cours de marché.

CMP recommandé

$$\text{CMP} = \frac{\text{Somme}[(\text{quantité restante du lot} \times \text{prix d'acquisition}) + \text{frais imputés}]}{\text{Somme}[\text{quantités restantes}]}$$

+/- Value

$$\text{+/- Value} = \text{Quantité détenue} \times (\text{Cours actuel} - \text{CMP}). \text{ La +/- Value en \%} = \text{+/- Value} / \text{Coût historique} \times 100.$$

Lors d'un achat, un nouveau lot est créé ou un lot agrégé est mis à jour selon la méthode retenue. Lors d'une vente, les quantités doivent être sorties selon une règle explicite (CMP, FIFO, LIFO ou méthode réglementaire applicable). Le choix doit être paramétré et documenté ; il ne doit pas être caché dans le frontend.

6. Base de marché : market_data

Les données de marché ont des fréquences et des volumes très différents des données clients. La base market_data peut utiliser PostgreSQL avec l'extension TimescaleDB pour les séries temporelles. Elle peut être alimentée par les API existantes de la plateforme et par les connecteurs de collecte.

Référentiels

Table	Rôle
exchanges	BRVM, NGX, GSE et autres places.
countries	Pays.
currencies	Devises et métadonnées.
issuers	Émetteurs.
sectors	Secteurs.
industries	Industries.
instruments	Référentiel instrument.
equities	Attributs propres aux actions.
bonds	Attributs propres aux obligations.

Table	Rôle
instrument_listings	Lien instrument ↔ place de cotation.

Actions et indices

Table	Rôle
equity_quotes	Dernière cotation / snapshot.
equity_ohlcv	Historique open-high-low-close-volume.
equity_order_book	Profondeur bid/ask.
equity_trades	Transactions de marché.
market_indices	Référentiel indices.
index_quotes	Historique des indices.

Obligations et courbes

Table	Rôle
bond_quotes	Cotations obligataires.
bond_ohlcv	Historique secondaire.
bond_cashflows	Coupons/amortissements théoriques.
bond_schedules	Échéancier détaillé.
bond_auctions	Adjudications / émissions.
yield_curves	Courbe par pays/date/méthode.
yield_curve_points	Points de courbe par maturité.

FX, calendrier et analyse

Table	Rôle
fx_rates	Taux de change.
corporate_actions	Splits, augmentations, opérations sur titres.
market_dividends	Dividendes annoncés.
trading_calendars	Jours ouverts et jours de bourse.
market_sessions	Heures et statut de séance.
technical_indicators	RSI, MACD, MM20/MM50 pré-calculés si utile.
financial_statements	États financiers.
fundamental_metrics	PER, ROE, ROA, P/B, marge, croissance, etc.

Tables à convertir en hypertables TimescaleDB

- equity_ohlcv
- bond_ohlcv
- equity_quotes
- equity_order_book

- equity_trades
- index_quotes
- fx_rates
- yield_curve_points

7. Zone d'intégration : data_integration

Cette zone protège les données officielles contre les erreurs provenant des API, fichiers Excel, imports SGI ou collectes web. Une donnée externe est d'abord reçue, contrôlée, rapprochée et seulement ensuite promue vers market_data ou gsm_portfolio.

Table	Rôle
data_sources	Référentiel des sources : API, Excel, SGI, bourse, dépôt manuel.
import_batches	Un lot d'import et son statut.
import_files	Fichiers attachés à un lot.
raw_records	Enregistrements bruts avant normalisation.
import_errors	Erreurs de parsing, mapping ou validation.
data_quality_checks	Résultats des contrôles qualité.
reconciliation_runs	Campagnes de rapprochement.
reconciliation_breaks	Écarts non résolus.
external_identifiers	Mapping identifiants source ↔ identifiants internes.
sync_jobs	Jobs de synchronisation.
sync_job_logs	Logs d'exécution et métriques.

Règle d'or

Une API ou un fichier Excel ne doit jamais écrire directement dans positions, position_lots ou portfolio_cash_accounts. La donnée passe par staging, validation et réconciliation.

8. Redis et stockage documentaire

Redis

- Cache des cotations et tableaux de bord.
- Broker/backend de tâches Celery.
- Verrous distribués pour certaines opérations concurrentes.
- Sessions ou jetons temporaires si l'architecture le requiert.
- Notifications temps réel / WebSocket.
- Calculs temporaires de simulation non encore validés.

Redis ne doit jamais être la source de vérité pour les positions, le cash, les transactions, le CMP ou les retraits.

Object Storage

Famille	Exemples
KYC / Client	Pièce d'identité, justificatifs, contrats.
Trading	Avis d'opéré, confirmations, bordereaux.
Reporting	Rapports PDF/Excel, rapports comité.
Retraits	Demandes signées, chèques, justificatifs de paiement.
Exports	Exports utilisateurs et snapshots documentaires.

La table documents conserve les métadonnées, la clé de stockage, le hash/checksum, le type MIME, le propriétaire, le statut et la date de création ; le binaire reste dans le stockage objet.

9. Correspondance entre les interfaces et les tables

Interface	Tables principales	Type d'interaction
Accueil	portfolio_valuations, portfolio_performance_snapshots, alerts, withdrawal_requests, positions + market_data	Lecture majoritaire ; agrégations et KPI.
Portefeuilles	clients, portfolios, investor_profiles, positions, portfolio_cash_accounts, portfolio_target_allocations	Lecture + création/modification contrôlée des fiches et mandats.
Portfeuille client	positions, position_lots, cash accounts, valuations, benchmarks, income events	CMP, +/- Value, allocations et historique.
Money Management	portfolio_cash_accounts, cash_movements, cash_reservations, expected_cashflows, liquidity_snapshots, liquidity_sources, liquidity_allocations	Lecture, affectation et réservation de liquidité.
Cession_Retrait	withdrawal_requests, withdrawal_plans, withdrawal_plan_lines, withdrawal_plan_versions, withdrawal_constraints	Création de la demande, génération et modification du plan.
Cession interne	internal_transfer_searches, candidates, matches, transfers, transfer_lines, settlements	Matching multi-gestionnaires et transfert validé.
Carnet d'ordres	orders, order_status_history, order_executions, order_allocations	Création, modification, annulation et suivi des ordres.
Avis d'opéré	trade_confirmations, transactions, executions, documents	Lecture et génération de confirmation.
Marchés Actions	market_data.exchanges, instruments, equity_quotes, equity_ohlcv, order_book, trades, indices	Lecture marché ; aucun stockage client.
Marchés Obligataire	bonds, bond_quotes, bond_ohlcv, cashflows, schedules, yield_curves	Lecture marché.
Watchlist	watchlists, watchlist_items + market_data	Création/suppression des instruments suivis.
Recommandations	investment_recommendations, recommendation_targets, allocation_recommendations, allocation_criteria	Création/validation/historisation des recommandations.
Alertes	alert_rules, alerts, alert_status_history, compliance_breaches	Traitement et clôture.
Rééquilibrage	target_allocations, positions, rebalancing_runs, lines, approvals, orders	Simulation puis génération d'ordres après validation.

Interface	Tables principales	Type d'interaction
Analyse portefeuille	performance_snapshots, risk_snapshots, stress_test_results, correlations	Lecture analytique, calculs batch.
Rapport comité	investment_committees, committee_reports, committee_decisions, impacts, report_documents	Génération, décision, suivi et audit.
Documentation	documents/report_documents ou fichiers statiques versionnés	Téléchargement des documents d'architecture et manuels.

10. Workflows transactionnels

10.1 Passage et exécution d'un ordre

10. Le frontend envoie un POST /orders avec portefeuille, instrument, sens, quantité et paramètres de prix.
11. Le backend vérifie utilisateur, mandat, conformité, quantité disponible ou cash disponible.
12. L'ordre est enregistré dans orders et son premier statut dans order_status_history.
13. Les exécutions reçues alimentent order_executions.
14. Chaque exécution crée une transaction et les frais associés.
15. Un achat crée/augmente les position_lots ; une vente consomme des lots selon la méthode retenue.
16. positions et portfolio_cash_accounts sont recalculés dans la même transaction ou via ledger contrôlé.
17. portfolio_valuations, allocations et alertes sont recalculées de façon synchrone ou asynchrone selon le besoin.

10.2 Cession interne liée à un retrait

18. Création de withdrawal_request.
19. Contrôle des règles bloquantes : montant > 95 % de l'encours ou solde final sous le minimum de maintien/clôture.
20. Création du withdrawal_plan et des lignes optimisées.
21. Modification possible du plan par le gestionnaire ; un plan insuffisant reste un avertissement non bloquant.
22. Recherche de contreparties internes dans les portefeuilles autorisés.
23. Contrôle de cash acheteur, profil, limites et conformité.
24. Validation de internal_transfer et de ses lignes.
25. À la date d'exécution : réduction des lots vendeur, création/augmentation des lots acheteur, mouvements de cash réciproques, frais et règlement.
26. Le reliquat non couvert reste à exécuter au marché.
27. Une fois tout le besoin de trésorerie disponible, withdrawal_payment finalise le retrait et l'historique est conservé.

11. Atomicité, concurrence et intégrité

Les opérations qui modifient simultanément cash, titres et statuts doivent être exécutées dans une transaction SQL unique. Le backend doit verrouiller les lignes critiques (par exemple SELECT ... FOR UPDATE sur comptes espèces, positions/lots et demandes de retrait) afin d'empêcher deux actions concurrentes de consommer les mêmes ressources.

Exemple : Valider une cession interne

Vérifier vendeur → vérifier titres disponibles → vérifier cash acheteur → vérifier conformité → verrouiller ressources →

créer transfert → enregistrer lignes → mettre à jour lots → produire mouvements espèces → recalculer positions → mettre à jour retrait → écrire audit → COMMIT. Une erreur à n'importe quelle étape = ROLLBACK complet.

Contraintes SQL essentielles

- Quantité disponible ≥ 0 et quantité bloquée ≥ 0 .
- Un ordre exécuté ne peut pas dépasser sa quantité totale.
- Un lot ne peut pas avoir $\text{quantity_remaining} > \text{quantity_initial}$.
- Une cession interne ne peut pas avoir vendeur = acheteur.
- Les devises doivent être cohérentes ou une conversion explicite doit être enregistrée.
- Les montants financiers utilisent NUMERIC/DECIMAL, jamais float.
- Les identifiants d'instruments doivent être stables et uniques par source/ISIN lorsque disponible.
- Les statuts sensibles doivent suivre une machine à états autorisée.

12. API métier : comment l'interface doit modifier la base

L'interface doit appeler des endpoints orientés métier, plutôt que des endpoints génériques permettant de modifier n'importe quelle colonne. Chaque endpoint applique les validations et déclenche les effets secondaires attendus.

Action interface	Endpoint indicatif	Effets principaux
Créer un client	POST /api/v1/clients/	clients + contacts + profil/mandat initial.
Créer un portefeuille	POST /api/v1/portfolios/	portfolio + comptes espèces/titres + allocation cible.
Passer un ordre	POST /api/v1/orders/	orders + contrôle pré-trade + réservation cash/titres.
Annuler un ordre	POST /api/v1/orders/{id}/cancel/	statut + libération des réservations.
Demander un retrait	POST /api/v1/withdrawals/	withdrawal_request + contrôles 95 % / minimum compte.
Générer plan retrait	POST /api/v1/withdrawals/{id}/plan/	plan + lignes + contraintes.
Modifier plan	PATCH /api/v1/withdrawal-plans/{id}/	nouvelle version + audit.
Valider cession interne	POST /api/v1/internal-transfers/	transfert + contrôles + réservations + audit.
Confirmer règlement	POST /api/v1/internal-transfers/{id}/settle/	transactions, lots, cash, statuts.
Traiter alerte	PATCH /api/v1/alerts/{id}/	statut + commentaire + historique.
Télécharger document	GET /api/v1/documents/{id}/download/ ou URL signée	contrôle autorisation + téléchargement.

13. Sécurité, audit et confidentialité

- RBAC : droits distincts pour administrateur, gestionnaire, opérateur, conformité, client.
- Scoping obligatoire : un gestionnaire ne voit que les clients/portefeuilles autorisés.
- Double validation possible pour transferts internes, retraits importants ou overrides conformité.
- Audit immuable ou append-only des actions sensibles.
- Chiffrement en transit (TLS) et au repos pour bases, sauvegardes et stockage documentaire.
- Masquage des données sensibles dans les logs.

- Sauvegardes automatisées, tests de restauration et politique de rétention.
- Idempotency keys pour les endpoints de paiement, règlement et création d'opérations critiques.

14. Noyau prioritaire : les 25 tables à implémenter d'abord

#	Table	Pourquoi en phase 1
1	users	Authentification et auteur des opérations.
2	sgis	Organisation propriétaire des comptes.
3	portfolio_managers	Gestionnaires.
4	clients	Référentiel client.
5	manager_client_assignments	Périmètre du gestionnaire.
6	investor_profiles	Profil risque.
7	portfolios	Conteneur métier central.
8	portfolio_target_allocations	Allocation cible.
9	portfolio_cash_accounts	Cash.
10	instruments	Référentiel titres.
11	positions	Détention courante.
12	position_lots	CMP et historique des acquisitions.
13	cash_movements	Ledger espèces.
14	cash_reservations	Blocage de cash.
15	orders	Ordres.
16	order_executions	Exécutions.
17	transactions	Mouvements titres/finance.
18	transaction_fees	Frais.
19	withdrawal_requests	Demande de retrait.
20	withdrawal_plans	Plan cession.
21	withdrawal_plan_lines	Lignes proposées.
22	internal_transfers	Cession interne.
23	internal_transfer_lines	Détail de la cession.
24	alerts	Alertes principales.
25	audit_logs	Traçabilité.

15. Plan de mise en œuvre recommandé

Phase	Contenu	Résultat
Phase 0 - Référentiel	Définir IDs, devises, bourses, instruments, SGI, utilisateurs et conventions comptables.	Aucun écran sensible ne doit démarrer sans référentiel stable.
Phase 1 - Portefeuille réel	Clients, portefeuilles, cash, positions, lots, CMP, transactions.	Portefeuille client et Présentation des actifs cotés deviennent réels.
Phase 2 - Trading	Ordres, exécutions, frais, règlement, avis	Carnet d'ordres et avis deviennent

Phase	Contenu	Résultat
	d'opéré.	persistants.
Phase 3 - Liquidité	Ledger cash, réservations, flux futurs, snapshots.	Money Management devient calculé depuis les écritures.
Phase 4 - Cession_Retrait	Demandes, plans, contraintes, cession interne, paiement.	Workflow complet et auditable.
Phase 5 - Risque / conformité	Alertes, limites, stress, approvals.	Prévention des violations et supervision.
Phase 6 - Reporting / comité	Rapports, décisions, impacts, documentation.	Gouvernance et historique documentaire.
Phase 7 - Industrialisation	Staging, réconciliation, monitoring, sauvegardes, performance.	Mise en production robuste.

16. Section Documentation dans l'interface

Le menu Gestionnaire doit comporter une entrée Documentation. Cette page peut devenir le point central des documents internes : architecture des bases, manuel utilisateur, procédures de Cession_Retrait, politiques de conformité, guides API et notes de version.

Élément	Comportement recommandé
Carte document	Titre, description, version, date, catégorie et type de fichier.
Télécharger	Télécharge le fichier depuis /public/documentation ou via un endpoint documentaire sécurisé.
Ouvrir	Prévisualise le PDF dans un nouvel onglet.
Version	Affiche la version et la date de mise à jour.
Production	Remplacer le fichier statique par documents + endpoint avec contrôle des permissions.

17. Checklist avant développement

- Valider le modèle de données et les relations cardinales.
- Valider la méthode de gestion des lots et du CMP.
- Valider les statuts des ordres, retraits et cessions internes.
- Définir le minimum de maintien/clôture par produit ou type de compte.
- Définir les frais et taxes par marché.
- Définir les droits et niveaux de validation par rôle.
- Lister les API de marché existantes à consommer et leurs fréquences.
- Définir les règles de réconciliation avec les SGI/dépositaires.
- Décider quelles données calculées sont persistées et lesquelles sont recalculées.
- Mettre en place audit, sauvegardes et monitoring avant la production.

18. Conclusion

L'objectif n'est pas de relier chaque composant React à une table isolée. Le logiciel doit être construit autour d'un modèle de données cohérent où les opérations métiers produisent les états affichés par les interfaces. Une transaction exécutée met à jour les lots, le CMP, la position, le cash, l'allocation et la valorisation ; une

demande de retrait déclenche un workflow traçable ; une cession interne affecte simultanément vendeur et acheteur ; une donnée de marché alimente la valorisation sans pouvoir modifier directement le portefeuille client.

Le noyau des 25 tables permet déjà de rendre opérationnels les écrans les plus importants. Les autres tables peuvent être ajoutées progressivement, domaine par domaine, sans casser l'architecture si les identifiants, les transactions et les frontières entre `gsm_portfolio`, `market_data` et `data_integration` sont correctement définis dès le départ.

Étape suivante recommandée

Transformer ce document en modèle relationnel exécutable : schéma ERD, modèles Django, migrations PostgreSQL, serializers/services DRF et matrice écran → endpoint → table → événements.